■ Java Servlets & JSP — Complete Study Notes

Covers: Architecture · Servlet Lifecycle · HTTP Methods · Navigation · Session · JSP Tags · Interview Q&A

Part 1 — Web Application Architecture

★ What is a Servlet?

- A managed Java class that EXTENDS server capabilities (not a standalone program)
- Has NO main() method — lives inside a Web Container (e.g. Apache Tomcat)
- Container handles: lifecycle, thread allocation, URL mapping, HTTP→Java conversion
- Implements the javax.servlet.Servlet interface (or extends HttpServlet)

■ Static vs Dynamic Content:

■ Static Content	■ Dynamic Content
• Plain HTML, CSS, JS files	• Personalized per user
• Served directly by web server	• Requires DB queries / logic
• Same for every user	• Needs a Servlet/JSP to process
• No business logic involved	• Generated at request time
• Fast — no processing needed	• Examples: login, cart, profile

■ Servlet Lifecycle (5 steps):

■ **service() is called per request — it internally routes to doGet() or doPost()**

Part 2 — HTTP Methods & Creating a Servlet

Core Servlet Code Structure:

```java
package com.example;

import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.*;

@WebServlet("/welcome")   // Maps to: localhost:8080/App/welcome
public class WelcomeServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
                         throws ServletException, IOException {

        response.setContentType("text/html");   // Tell browser what's coming
        PrintWriter out = response.getWriter(); // Open output stream

        String name = request.getParameter("username"); // Read form data
        if (name == null) name = "Guest";

        out.println("<html><body>");
        out.println("<h1>Hello, " + name + "!</h1>");
        out.println("</body></html>");
    }
}
```

GET vs POST:

■ GET Method	■ POST Method
• Data appended to URL as ?key=val	• Data hidden in HTTP request body
• Visible in browser address bar	• Not visible in address bar
• NOT secure (passwords exposed!)	• SECURE — ideal for passwords
• URL length limit ~2048 chars	• No size limit (files, blobs OK)
• Bookmarkable / cacheable	• Not bookmarkable
• Use: search, page navigation	• Use: login, form submit, upload
• Override: doGet(req, res)	• Override: doPost(req, res)

■■ Key API methods to remember:

- request.getParameter("key") → Read a single form field value
- request.getParameterValues("key") → Read multi-select / checkbox array
- response.setContentType("text/html") → Set MIME type before writing
- response.getWriter() → Get PrintWriter to write HTML back
- response.setStatus(404) → Set HTTP status code manually

Part 3 — Server Navigation & State Management

■ RequestDispatcher (Forward) vs sendRedirect:

■ Forward (RequestDispatcher)

- Internal server-side transfer
- Browser URL stays THE SAME
- 1 HTTP request total
- request attributes PRESERVED
- request.setAttribute(k, v) works
- Faster (no network round-trip)
- Use: forward to JSP view

■ sendRedirect

- Server sends 302 to browser
- Browser URL CHANGES to new path
- 2 HTTP requests total
- request attributes are LOST
- New request = fresh scope
- Slower (browser makes new req)
- Use: after form POST (PRG pattern)

Code Examples:

```
// ■■ Forward (RequestDispatcher) ■■
request.setAttribute("user", userObject);   // Store data in request scope
RequestDispatcher rd = request.getRequestDispatcher("/result.jsp");
rd.forward(request, response);              // Pass to JSP — URL unchanged

// ■■ sendRedirect ■■
response.sendRedirect("/login.jsp");        // 302 → browser makes new GET
response.sendRedirect("https://google.com"); // Can redirect to external URL

// ■■ Reading forwarded attribute in JSP ■■
String user = (String) request.getAttribute("user"); // Retrieve in JSP
```

■ Session Management — HTTP is Stateless!

> **HTTP is stateless — every request is treated independently with no memory of past requests**

■ Cookies

- Key-value pairs in browser
- Server sends via Set-Cookie header
- Browser returns on every request
- Client-side storage (less secure)
- Can set expiry / max-age
- New Cookie("key","val") to create
- response.addCookie(cookie)

■ HttpSession

- Server-side memory storage
- Linked via JSESSIONID cookie
- request.getSession() to create
- session.setAttribute(k, v)
- session.getAttribute(k) to read
- session.invalidate() to logout
- More secure than cookies

```
// ■■ Cookie creation & sending ■■
Cookie c = new Cookie("username", "Alice");
c.setMaxAge(60 * 60 * 24);  // 1 day in seconds
response.addCookie(c);      // Send to browser

// ■■ Reading a cookie ■■
Cookie[] cookies = request.getCookies();
for (Cookie ck : cookies) {
    if (ck.getName().equals("username")) { String val = ck.getValue(); }
}

// ■■ HttpSession usage ■■
HttpSession session = request.getSession();  // Create or retrieve
session.setAttribute("cart", cartObject);    // Store any Java object
Object cart = session.getAttribute("cart");  // Retrieve later
session.invalidate();                        // Destroy session (logout)
```

■ JSESSIONID Explained:

When getSession() is called, Tomcat creates a session with unique ID (JSESSIONID). This ID is sent to the browser as a cookie. On subsequent requests, the browser sends JSESSIONID back, letting Tomcat find the right session object in memory.

Part 4 — JavaServer Pages (JSP)

★ Why JSP?

Problem: Writing HTML inside Servlet = endless out.println("<html>...") = messy!

Solution: JSP reverses this — write Java INSIDE an HTML file (.jsp extension)

Benefit: Clean separation, easy to read, web designers can work on it too

■ JSP Lifecycle (first request only, then steps 1-2 are skipped):

*** Steps 1-2 happen only ONCE on first request. Subsequent requests jump to step 4 directly.**

■ JSP Scripting Elements — All 4 Tags:

1■■ Scriptlet Tag <% ... %> — Java logic

- Write multi-line Java: conditions, loops, variable declarations
- Code executes inside _jspService() method (per request)
- Can use 'out' object directly (implicit object)

```
<%
    int status = 1;
    if (status == 1) {
        out.println("<p>Status: Active</p>");   // 'out' is implicit
    }
    for (int i = 0; i < 5; i++) {
        out.println("<li>Item " + i + "</li>");
    }
%>
```

2■■ Expression Tag <%= ... %> — Print a value ■■ NO semicolon!

- Shorthand to output values — acts like out.print(value)
- NEVER put a semicolon inside — it will cause a compilation error!
- Can output variables, method calls, or calculations

```
<p>Result: <%= (5 * 10) %></p>        <!-- outputs: Result: 50 -->
<p>User: <%= username %></p>          <!-- outputs variable value -->
<p>Date: <%= new java.util.Date() %></p>  <!-- method call OK too -->

<!-- WRONG: <%= x; %>  |  CORRECT: <%= x %> -->
```

3■■ Declaration Tag <%! ... %> — Define methods/fields

- Declares instance variables or methods OUTSIDE _jspService()
- Shared across ALL requests (unlike scriptlet variables which are per-request)
- Use for helper methods, counters, constants

```
<%!                                        // Instance-level declaration
    private int requestCount = 0;          // Shared across requests

    public int multiplyByTwo(int value) {  // Reusable helper method
        return value * 2;
    }
%>

<p>Count: <%= ++requestCount %></p>        // Use declared field
<p>Result: <%= multiplyByTwo(7) %></p>     // Call declared method
```

4■■ Directive Tag <%@ ... %> — Compiler instructions

- Sends global settings to the JSP compiler/translator
- 3 types: page, include, taglib
- Processed at TRANSLATION time (not runtime)

```
<%@ page import="java.util.ArrayList" %>      // Import classes
<%@ page import="java.util.Collections" %>
<%@ page contentType="text/html" pageEncoding="UTF-8" %>
<%@ page errorPage="error.jsp" %>             // Custom error page
<%@ page session="false" %>                   // Disable session tracking

<%@ include file="header.jsp" %>              // Static include at translate time
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>  // JSTL
```

Part 5 — JSP Implicit Objects (Auto-available in JSP)

Object	Type	Purpose
`out`	JspWriter	Write output to browser (like PrintWriter)
`request`	HttpServletRequest	Access request params, attributes, headers
`response`	HttpServletResponse	Modify response (status, headers, redirect)
`session`	HttpSession	Store/retrieve session-scoped data
`application`	ServletContext	App-wide data shared across all users/sessions
`config`	ServletConfig	Servlet config & init parameters
`pageContext`	PageContext	Access to all other implicit objects
`page`	Object	Reference to current JSP page (like 'this')
`exception`	Throwable	Only in error pages — holds the exception

```
// Common JSP implicit object usage examples:
String name = request.getParameter("name");   // Read form input
session.setAttribute("loggedUser", name);     // Save in session
String user = (String) session.getAttribute("loggedUser");
application.setAttribute("visitCount", 100); // App-wide counter
out.println("<h1>Hello</h1>");               // Write HTML output
response.sendRedirect("home.jsp");           // Redirect from JSP
```

Part 6 — Quick Reference Cheat Sheet

Tag / API	Syntax / Method	Use For
`Scriptlet`	`<% ... %>`	Java logic, conditions, loops
`Expression`	`<%= value %>`	Print output (NO semicolon!)
`Declaration`	`<%! ... %>`	Methods & instance fields
`Directive`	`<%@ page/include/taglib %>`	Imports, config, error pages
`Get param`	`request.getParameter("k")`	Read HTML form field
`Set attr`	`request.setAttribute(k,v)`	Pass data to forwarded resource
`Get attr`	`request.getAttribute(k)`	Read forwarded data in JSP
`Forward`	`dispatcher.forward(req,res)`	Server-side handoff, URL same
`Redirect`	`response.sendRedirect(url)`	302 browser redirect, URL changes
`Create sess`	`request.getSession()`	Get/create HttpSession object
`Session set`	`session.setAttribute(k,v)`	Store in session
`Session get`	`session.getAttribute(k)`	Retrieve from session
`Invalidate`	`session.invalidate()`	Destroy session (logout)
`Cookie`	`new Cookie("k","v")`	Create a new cookie
`Send cookie`	`response.addCookie(c)`	Send cookie to browser
`Content type`	`response.setContentType(t)`	Set MIME type of response
`Writer`	`response.getWriter()`	Get PrintWriter for output

Part 7 — Top Interview Questions & Answers

Q1 What is a Servlet? How is it different from a normal Java class?

Answer:

A Servlet is a Java class that handles HTTP requests/responses inside a Web Container.
Unlike normal classes, it has no main() method. The container manages its lifecycle.
It extends HttpServlet and is mapped to URLs via @WebServlet annotation.

Q2 Explain the Servlet lifecycle in order.

Answer:

1. Class loading 2. init() — called once when servlet is first loaded
3. service() — called for every request, routes to doGet/doPost
4. destroy() — called once before servlet is removed from memory
5. Garbage collection — container releases the object

Q3 What is the difference between doGet() and doPost()?

Answer:

doGet(): data visible in URL, limited size, used for search/navigation, cacheable
doPost(): data in request body, secure, no size limit, used for login/file upload
Override both in your servlet to handle either type of request

Q4 What is the difference between RequestDispatcher.forward() and sendRedirect()?

Answer:

forward(): server-side, URL stays same, 1 request, request attributes preserved
sendRedirect(): browser-side 302, URL changes, 2 requests, attributes lost
Use forward to pass to JSP view; redirect after POST to prevent re-submission

Q5 What is a session? How does HttpSession work?

Answer:

Session = server-side storage that persists across multiple requests from same user
Tomcat creates a unique JSESSIONID and sends it as a cookie to the browser
Browser returns JSESSIONID on each request; Tomcat finds the matching session

Q6 What is the difference between Cookies and HttpSession?

Answer:

Cookies: stored in browser (client-side), less secure, can expire, size limited
HttpSession: stored in server memory, more secure, linked via JSESSIONID cookie
Sessions end when browser closes or session.invalidate() is called

Q7 What is JSP? How is it different from a Servlet?

Answer:

JSP = HTML file with embedded Java (Java inside HTML)
Servlet = Java class writing HTML (HTML inside Java — messy!)
JSP is compiled into a Servlet by the container. Both produce the same output.

Q8 Explain the JSP lifecycle.

Answer:

1. Translation: .jsp → .java (Servlet source code)
2. Compilation: .java → .class (bytecode)
3. Loading + init() 4. _jspService() called per request 5. destroy()

Steps 1-2 happen ONCE on first request only

Q9 What are the 4 JSP scripting elements? Give the syntax for each.

Answer:

Scriptlet: <% java code %> — logic, loops, conditions

Expression: <%= value %> — print output (NO semicolon!)

Declaration: <%! method or field %> — define methods/instance vars

Directive: <%@ page/include/taglib %> — compiler instructions

Q10 Name any 5 JSP implicit objects and their types.

Answer:

out (JspWriter), request (HttpServletRequest), response (HttpServletResponse)

session (HttpSession), application (ServletContext)

Also: config, pageContext, page, exception (error pages only)

Q11 What is the difference between <%! %> and <% %> in JSP?

Answer:

<% %> Scriptlet: code goes inside _jspService() — local, per-request scope

<%! %> Declaration: code goes OUTSIDE _jspService() — instance level, shared

Variables in scriptlet are re-created each request; declarations are shared

Q12 What is @WebServlet annotation? What was used before it?

Answer:

@WebServlet("/url") maps the servlet to a URL pattern (since Servlet 3.0)

Before Servlet 3.0, mapping was done in web.xml using <servlet> and <servlet-mapping>

Annotation approach is cleaner and avoids XML configuration

Q13 How do you pass data from Servlet to JSP?

Answer:

Use request.setAttribute("key", value) in the Servlet

Then forward using RequestDispatcher: dispatcher.forward(request, response)

In JSP read with: (Type) request.getAttribute("key")

Q14 What is the web.xml file? Is it mandatory?

Answer:

web.xml is the Deployment Descriptor — configures servlets, mappings, filters

Since Servlet 3.0 with @WebServlet annotation, web.xml is OPTIONAL

Still needed for advanced config: welcome files, error pages, security roles

Q15 What is the difference between include directive and include action in JSP?

Answer:

<%@ include file="header.jsp" %> — static include at TRANSLATION time (merged)

<jsp:include page="header.jsp" /> — dynamic include at REQUEST time

Directive includes once at compile; action re-processes each request

Final Page — Architecture Overview & Memory Aids

■ Full Request Flow:

■ Memory Tricks:

- ❖ GET = Give Everyone Text (visible in URL)
- ❖ POST = Protected, Opaque, Secure Transfer
- ❖ Forward = 'F' for Fixed URL (same URL stays)
- ❖ Redirect = 'R' for Route-change (URL changes)
- ❖ JSP lifecycle: T-C-L-E-D = Translate, Compile, Load, Execute, Destroy
- ❖ <%= no semicolon! %> — Expression = Equals sign = just a value, never a statement
- ❖ JSESSIONID = J-Session-ID = Java Session Identifier cookie

★ Formula Strip

```
Servlet     = Java class inside Container (no main, doGet/doPost are entry points)
JSP         = HTML + Java  →  compiled INTO a Servlet by Tomcat automatically
Forward     = same request, same URL    |  Redirect = new request, new URL
Cookie      = client-side storage       |  HttpSession = server-side storage
Scriptlet   = <% logic %>               |  Expression = <%= value %> (no ;)
Declaration= <%! method %>              |  Directive  = <%@ config %>
```

Topics mastered: Architecture · Lifecycle · GET vs POST · Forward vs Redirect · Cookies vs Sessions · JSP Tags · Implicit Objects · Interview Q&A ★ ★ ★